

Настройка celery в Django

1. Создаем файл `celery.py` рядом с файлом `settings.py` со следующим содержимым

```
import os

from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "project.settings")

app = Celery("celery")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()
```

Путь к модулю настроек

2. Рядом с `settings.py` в файле `__init__.py` добавляем:

```
from .celery import app

__all__ = ["app"]
```

3. Далее в приложении Django создаем файл или пакет **tasks** и в него добавляем задачи:

```
@shared_task
def common_shared_task(var1: str, var2: str):
    time.sleep(3)
    print("Hello from celery shared_task", var1, var2)
    return f"shared_task {var1}, {var2}".upper()
```

4. В файле `setting.py` добавляем настройки celery, не забываем про префикс **CELERY**

```
# Брокер сообщений: Используется Redis для распределения задач между воркерами.
CELERY_BROKER_URL = os.getenv("CELERY_BROKER_URL", "redis://127.0.0.1:6379/1")

# Результаты задач: Сохраняются в отдельной Redis-базе.
CELERY_RESULT_BACKEND = os.getenv("CELERY_RESULT_BACKEND", "redis://127.0.0.1:6379/2")

# Маршрутизация задач:
CELERY_TASK_ROUTES = {
    "posts.tasks.*": {"queue": "posts"}, # Задачи из модуля posts.tasks направляются в очередь posts.
}

# Используется JSON для передачи данных задач и их результатов.
CELERY_TASK_SERIALIZER = "json"
CELERY_RESULT_SERIALIZER = "json"

# В режиме разработки (dev) задачи выполняются немедленно, что удобно для отладки.
CELERY_TASK_ALWAYS_EAGER = ENV == "dev"

# Установлены таймауты и ограничения памяти для воркеров, чтобы избежать утечек ресурсов.
CELERY_TIME_LIMIT = 60 * 60
CELERY_WORKER_MAX_MEMORY_PER_CHILD = 1024 * 1024 * 300 # 300 MB
```